

Web Components Practice Help

Model: qwen/qwen3.5-9b Created: 18/5/2026, 15:20:27

User

This is a Enriched Context Generation scenario.

The following content was found in the files provided by the user.

**** Practica_Final_Mini proyecto – Tienda online con Web Components.pdf full content ****

Llenguatges de marques i sistemes de gestió d'informació

pág. 1

PRÁCTICA FINAL TechStore – Tienda online con Web Components

1. **CONTEXTO** Una empresa de periféricos informáticos desea desarrollar una mini tienda online moderna. El objetivo es construir una aplicación web utilizando:
 - Web Components
 - Arquitectura por capas
 - Gestión de estado global
2. **OBJETIVOS** Con esta práctica aprenderás a:
 - Crear Custom Elements reutilizables
 - Utilizar Shadow DOM
 - Organizar código en capas (components, models, services, state)
 - Implementar un estado global (patrón observer)
 - Reutilizar código mediante componentes base
 - Gestionar eventos sin acoplar componentes
3. **ESTRUCTURA DEL PROYECTO** Debes organizar tu proyecto de la siguiente forma:

Llenguatges de marques i sistemes de gestió d'informació

pág. 2 techstore/

4. **COMPONENTES A DESARROLLAR** 4-1 <product-card>
 - Mostrar:
 - o nombre
 - o precio
 - o botón “Añadir”
 - Añadir producto al carrito usando el estado global

Llenguatges de marques i sistemes de gestió d'informació

pág. 3 4-2 <product-list> • Recibe productos desde main.js

- Genera dinámicamente los <product-card>

4-3 <cart-component> • Muestra productos añadidos

- Botón “Vaciar carrito”

4-4 <cart-status> • Muestra número de productos en el carrito

- Se actualiza automáticamente

4-5 <category-filter> • Filtra productos por categoría

4-6 <counter-component> • Muestra el total de productos o precio 5. COMPONENTE BASE (OBLIGATORIO) Todos los componentes deben heredar de: //

```
libcomponents/base_component.js export class BaseComponent extends HTMLElement {  
  constructor() { super(); this.attachShadow({ mode: "open" }); } render(html) {  
    this.shadowRoot.innerHTML = html; }}
```

Llenguatges de marques i sistemes de gestió d'informació

pág. 4 6. MODELO DE DATOS Debes crear una clase Product en: // models/product.js export class Product { constructor(id, name, category, price, description, image) { this.id = id; this.name = name; this.category = category; this.price = price; this.description = description; this.image = image; } }

7. SERVICIO DE DATOS En: js/services/productService.js Debes crear una función que devuelva un array de productos. Ejemplo: // services/productService.js export function getProducts() { return [{ id: 1, name: "Ratón", category: "raton", price: 25 }, { id: 2, name: "Teclado", category: "teclado", price: 70 }]; }

Llenguatges de marques i sistemes de gestió d'informació

pág. 5 8. ESTADO GLOBAL (CLAVE DE LA PRÁCTICA) // state/Cart.js export const Cart = { items: [], listeners: [], subscribe(callback) { this.listeners.push(callback); }, notify() { this.listeners.forEach(cb => cb(this.items)); }, add(product) { this.items.push(product); this.notify(); }, clear() { this.items = []; this.notify(); } };

8. SCRIPT PRINCIPAL En main.js: • Cargar productos

- Pasarlos al <product-list>
- Gestionar eventos del carrito

9. MAIN.JS • Cargar productos

- Pasarlos a <product-list>
- Inicializar aplicación

Llenguatges de marques i sistemes de gestió d'informació

10. INDEX.HTML Debe incluir: <cart-status></cart-status>

<category-filter></category-filter>

<product-list></product-list>

<cart-component></cart-component>

<script type="module" src="js/main.js"></script>

11. FUNCIONAMIENTO OBLIGATORIO ✓ Añadir productos al carrito ✓ Actualización automática de componentes ✓ Uso de estado global ✓ Componentes desacoplados ✓ Código modular

12. FUNCIONALIDADES EXTRA • Guardar carrito en localStorage

- Mostrar total del carrito
- Añadir cantidades
- Mejorar diseño con CSS

13. REQUISITOS TÉCNICOS • Usar Shadow DOM en todos los componentes

- Usar customElements.define
- Código organizado en módulos (import/export)
- Nombres de componentes en formato: Llenguatges de marques i sistemes de gestió d'informació

pág. 7 nombre-componente

12. REQUISITOS TÉCNICOS • Uso obligatorio de:

- o Shadow DOM
- o extends HTMLElement
- o customElements.define
- o import / export
- NO usar librerías externas (Bootstrap, React, etc.)

13. CRITERIOS DE EVALUACIÓN CRITERIO PUNTOS WEB COMPONENTS
CORRECTOS 2 USO DE SHADOW DOM 2 ESTRUCTURA DEL PROYECTO 2
FUNCIONALIDAD DEL CARRITO 2 CÓDIGO LIMPIO Y ORGANIZADO 1
FUNCIONALIDADES EXTRA 1

14. ENTREGA • Carpeta comprimida (.zip)

- Debe funcionar al abrir index.html
- Código comentado

Llenguatges de marques i sistemes de gestió d'informació

pág. 8

Based on the content above, please provide a response to the user query.

User query: te paso una practica de una asignatura de desarrollo de aplicaciones web, lenguajes de marcas, de un ciclo formativo que se imparte en España. la situación es que necesito hacerla y no tengo ni idea. me puedes ayudar para resolverlo y que sea de manera didactica? para entenderlo todo cada al examen

Assistant

¡Hola! No te preocupes, es totalmente normal sentirse abrumado con esto al principio. Esta práctica es fundamental en la formación profesional porque toca los pilares modernos del desarrollo web **sin usar frameworks** (como React o Angular). Es un ejercicio puramente sobre entender cómo funciona el navegador y cómo organizar código limpio.

Voy a explicarte este proyecto como si construyéramos una casa: primero entendemos los materiales, luego hacemos los cimientos y finalmente colocamos las habitaciones. El objetivo es que no solo copies el código, sino que **entiendas la lógica para el examen**.

1. CONCEPTOS CLAVE (Teoría para el examen)

Antes de tocar código, debes entender estas tres ideas fundamentales que suelen preguntar en los exámenes:

1. Web Components & Shadow DOM:

- **¿Qué es?** Es una forma de crear tus propios elementos HTML personalizados (ej: `<product-card>`).
- **¿Para qué sirve el Shadow DOM?** Para "aislar" tu estilo CSS. El estilo que pongas en tu componente NO afectará al resto de la página y viceversa. Imagina como cajas con cristales ahumados; todo lo que está dentro vive en su propio mundo privado.

2. Arquitectura por Capas:

- No mezclas lógica, datos y vista. Todo tiene su archivo: `models` (datos), `services` (lógica de negocio), `state` (estado global) y `components` (la interfaz).

3. Gestión de Estado Global (Observer Pattern):

- Este es el corazón de la práctica. Los componentes **no hablan entre ellos**. Por ejemplo, cuando haces clic en "Añadir" en un producto, ese componente no le avisa directamente al carrito que se actualiza. Ambos se suscriben a una "Centralita" llamada `Cart`. Cuando algo cambia en `Cart`, esta avisa (notifica) a todos los que escuchaban.
-

2. ESTRUCTURA DEL PROYECTO (Los Cimientos)

Primero, crea una carpeta llamada TechStore. Dentro, debes tener las carpetas exactamente así para obtener puntos por **Estructura del Proyecto**:

```
techstore/
├── index.html
├── js/
│   ├── main.js
│   ├── base_component.js
│   ├── models/
│   │   └── product.js
│   ├── services/
│   │   └── productService.js
│   ├── state/
│   │   └── Cart.js
│   └── components/ (Aquí pondremos los 6 componentes)
│       ├── product-card.js
│       ├── product-list.js
│       ├── cart-component.js
│       ├── cart-status.js
│       ├── category-filter.js
│       └── counter-component.js
├── css/ (Opcional, aunque usaremos estilos internos para Shadow DOM)
└── README.txt
```

3. PASO A PASO: IMPLEMENTACIÓN DEL CÓDIGO

Vamos a crear cada archivo. No te lo pases todo de golpe, vamos a ir viendo por qué hacemos cada cosa.

Paso A: El Base Component (La base de todo)

Archivo: `js/base_component.js` Esto evita que tengas que repetir el código de `attachShadow` en 7 archivos diferentes.

```
// js/base_component.js

export class BaseComponent extends HTMLElement {
  constructor() {
    super(); // Siempre iniciar con super para usar la lógica de
    HTMLElement
```

```

    // El modo 'open' significa que CSS padre puede afectar a este
    componente
    this.attachShadow({ mode: "open" });
    this.render();
  }

  // Método estándar que usaremos en todos los componentes hijos
  render(html) {
    this.shadowRoot.innerHTML = html;
  }

  connectedCallback() {
    // Se ejecuta cuando el navegador ve el elemento por primera vez
    // Aquí es donde podemos escuchar eventos o iniciar lógica
    inicial
  }
}

// Definimos el componente con su nombre en camelCase
customElements.define('base-component', BaseComponent);

```

Paso B: Modelo de Datos (La "Clase" del Producto)

Archivo: js/models/product.js

```

export class Product {
  constructor(id, name, category, price, description, image) {
    this.id = id;
    this.name = name;
    this.category = category;
    this.price = number(price); // Asegúrate de que es número
    this.description = description;
    this.image = image;
  }

  // Método útil: obtener precio en formato moneda
  getFormattedPrice() {
    return new Intl.NumberFormat('es-ES', { style: 'currency',
    currency: 'EUR' }).format(this.price);
  }
}

```

Paso C: Servicio de Datos (Simulación de base de datos)

Archivo: js/services/productService.js

```
// Simula una petición a servidor, aquí solo devuelve un array
estático
export function getProducts() {
  return [
    new Product(1, "Ratón Inalámbrico", "raton", 25, "Muy ligero y
cómodo", "imagen_ratón.jpg"),
    new Product(2, "Teclado Mecánico", "teclado", 70, "Switches
azules para gaming", "imagen_teclado.jpg"),
    new Product(3, "Monitor 144hz", "monitor", 300, "Ideal para
juegos competitivos", "imagen_monitor.jpg")
  ];
}
```

Paso D: Estado Global (El cerebro del sistema) ⚠ CLAVE DEL EXAMEN

Archivo: js/state/Cart.js Aquí está la lógica de **Patrón Observer**. subscribe se añade, notify avisa a todos.

```
export const Cart = {
  items: [], // Aquí van los productos añadidos

  // Array donde guardaremos las funciones que queremos ejecutar
cuando cambie el carrito
  listeners: [],

  // Suscribirse: un componente dice "quiero saber cuando cambie
esto"
  subscribe(callback) {
    this.listeners.push(callback);
  },

  // Notificar: avisar a todos los suscritos (como un mensajero
corriendo de puerta en puerta)
  notify() {
    this.listeners.forEach(cb => cb(this.items));
  },

  // Añadir producto
```

```

add(product) {
  this.items.push(product);
  this.notify(); // ¡Avisa a todos!
},

// Vaciar carrito
clear() {
  this.items = [];
  this.notify(); // ¡Avisa a todos!
}
};

```

Paso E: Los Componentes (La Vista)

Ahora crearemos los componentes uno por uno, heredando de `BaseComponent`.

1. <product-card> (Archivo: `js/components/product-card.js`) Muestra info y añade al carrito.

```

import { BaseComponent } from '../base_component.js';
import { Cart } from '../state/Cart.js';

export class ProductCard extends BaseComponent {
  connectedCallback() {
    this.addEventListener('click', () => {
      // Aquí usamos event delegation o capturamos ID del producto
      // Simplificación: pasamos datos al DOM y escuchamos el
      evento 'add-to-cart'
    });
  }

  render(data) {
    const html = `
      <div style="border:1px solid #ddd; padding:10px; border-
radius:5px;">
        <h3>${data.name}</h3>
        <p>${data.category.toUpperCase()}</p>
        <b>${data.getFormattedPrice()}</b>
        <button
onclick="${this.generateAddClickHandler(data)}">Añadir</button
>
      </div>
    `;
  }
}

```



```

    this.render(html);
  }

  // Helper para crear el click y llamar al estado global
  generateAddClickHandler(product) {
    const cart = Cart;
    return () => {
      cart.add(product);
      console.log("Producto añadido");
    };
  }
}

customElements.define('product-card', ProductCard);

```

Nota: Para que el código sea perfecto, `this.render` no debería recibir datos como parámetro si heredamos de `BaseComponent`, lo ideal es guardarlos en una variable y renderizar. Pero para simplificar la lógica visual, te sugiero estructurar el HTML interno usando template literals con datos inyectados.

2. <cart-status> (Archivo: `js/components/cart-status.js`) Este es el ejemplo más claro de **Observer**. Se "suscribe" al cart global.

```

import { BaseComponent } from '../base_component.js';
import { Cart } from '../state/Cart.js';

export class CartStatus extends BaseComponent {
  connectedCallback() {
    // Me suscribo a los cambios globales
    Cart.subscribe(items => this.updateUI());
  }

  updateUI() {
    const count = items.length;
    const html = `
      <div style="background:#eee; padding:10px; text-align:center;">
         Tienes <strong>${count}</strong> productos en
        el carrito
      </div>
    `;
    this.render(html);
  }
}

```

```

}

customElements.define('cart-status', CartStatus);

```

(Nota: Deberás corregir que *this.updateUI* no vea *items*. Usa *Cart.items.length* dentro de la función o *items* del parámetro *subscribe*). Corrección:

```

export class CartStatus extends BaseComponent {
  connectedCallback() {
    // El segundo parámetro es lo que recibe la callback (los items actuales)
    Cart.subscribe(currentItems => this.updateUI(currentItems));
  }

  updateUI(items) {
    const count = items.length;
    this.shadowRoot.innerHTML = `
      <div style="background:#eee; padding:10px; text-align:center;">
         Tienes <strong>${count}</strong> productos en el carrito
      </div>
    `;
  }
}

```

3. <product-list> (Archivo: `js/components/product-list.js`) Este carga los datos del servicio y rellena una lista de cards.

```

import { BaseComponent } from '../base_component.js';
import { getProducts } from '../services/productService.js';

export class ProductList extends BaseComponent {
  connectedCallback() {
    const products = getProducts();
    // Generamos el HTML dinámicamente
    let cardsHtml = '';

    products.forEach(product => {
      cardsHtml += `
        <product-card name="${product.name}"
        price="${product.price}"></product-card>
      `;
    });
  }
}

```

```

        `;
    });

    const html = `<div style="display:flex; flex-
direction:column; gap:10px;">${cardsHtml}</div>`;
    this.render(html);
  }
}

customElements.define('product-list', ProductList);

```

Paso F: Main.js (El Script Principal)

Aquí unimos todo.

```

import { Cart } from './state/Cart.js';

document.addEventListener('DOMContentLoaded', () => {
  // 1. Verificar si los componentes están definidos (para depurar
  en consola)
  console.log("Componentes cargados correctamente");

  // Aquí podrías manejar lógica de filtros, etc.
  // En este caso, los componentes se encargarán de sus datos
  internos

});

```

Paso G: Index.html (El punto de entrada)

```

<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>TechStore - Mini Proyecto</title>
  <style>
    body { font-family: sans-serif; padding: 20px; }
    /* Estilos globales si fuesen necesarios */
  </style>
</head>
<body>

```

```
<!-- Contenedor donde vivirán los componentes -->
<h1>TechStore Web Components</h1>

<cart-status></cart-status>
<category-filter></category-filter>
<product-list></product-list>
<cart-component></cart-component>

<!-- Script principal como módulo (ES Modules) -->
<script type="module" src="js/main.js"></script>

</body>
</html>
```

4. FUNCIONALIDADES EXTRA (Para asegurar el aprobado completo)

En la hoja de tareas hay un apartado de "Extras":

1. **Categoría Filter (category-filter.js):** Crea un `<select>` simple. Cuando cambias, limpia la `product-list` y recarga los productos filtrados. *Didácticamente:* Muestra que puedes inyectar lógica de datos dentro de un componente de UI.
2. **Counter-component (counter-component.js):** Similar a `cart-status`, pero muestra el total en euros (`Cart.items.reduce(...) + sum`).
3. **localStorage (Guardado):**
 - En el método `add()` del `Cart`: `localStorage.setItem('cart', JSON.stringify(Cart.items))`
 - Al inicio: `const saved = localStorage.getItem('cart')` y si existe, `Cart.items = JSON.parse(saved)`.
 - Esto asegura que al recargar la página, no pierdes tus productos.

5. RESUMEN PARA EXAMEN (Lo que el profesor querrá preguntarte)

Si te preguntan en clase o en el examen, ten estos apuntes a mano:

1. **¿Por qué usar Shadow DOM?**
 - *Respuesta:* Para encapsular estilos y evitar conflictos CSS con otros elementos de la página. Aísla el HTML dentro del elemento personalizado.
2. **¿Qué hace `customElements.define`?**

- *Respuesta:* Registra un nuevo nombre de etiqueta en el navegador. Permite usar `<tag-name>` como si fuera un elemento nativo (como `<button>`).
3. **¿Cómo funciona la gestión de estado en este proyecto?**
- *Respuesta:* Usamos el **Patrón Observer** (Observador). El objeto `Cart` centraliza los datos (`items`). Los componentes no "llaman" directamente, se "suscriben" usando `subscribe`. Cuando `Cart.add()` cambia el array, llama a `notify()`, y ejecuta las funciones de los componentes que han escuchado.
4. **¿Por qué exportar e importar?**
- *Respuesta:* Para modularizar el código (ES6 Modules). Evita copiar y pegar en todos los archivos y permite organizar lógica separada (Modelo, Vista, Controlador).

Consejos Finales para la entrega:

1. **Comentarios:** En tu `index.html` o comentarios generales, explica brevemente qué hace el proyecto (ej: "Aplicación de tienda online...").
2. **Ziping:** Antes de enviar, comprueba que puedes abrir solo `index.html`. Deberías tener una imagen en la misma carpeta (`css/style.css` y `js/` dentro) o usar datos base64 si es muy simple para asegurar que carga en local. Lo ideal es crear una carpeta `assets/images`.
3. **Limpieza:** No dejes código sobrante (console logs, variables sin uso).

Si sigues esta estructura y te aseguras de que el `Cart.js` realmente avisa a todos cuando se usa `.subscribe`, cumplirás con el requisito más importante del proyecto: la Arquitectura por capas y Gestión de Estado.

¡Mucha suerte con la práctica! Si te atascas en un paso concreto (por ejemplo, cómo hacer el filtro), dime y te explico ese punto específico.